MOVIE RECOMMENDATION BY LOW-RANK FACTORIZATION · MOVIELENS 100K

Elaborato M3–M4

Fattorizzazione matriciale con SGD, scelta degli iperparametri e valutazione finale

1 Dalle baseline ai fattori latenti
2 Il modello
3 L'addestramento con SGD
4 Analisi e interpretazione dei risultati
5 Valutazione finale sul test set

1 Dalle baseline ai fattori latenti

Nei punti 1 e 2 abbiamo visto che la matrice utenti × film di MovieLens 100K è **quasi vuota**: 943 utenti e 1682 film fanno circa 1,6 milioni di caselle possibili, ma i voti osservati sono 100.000, cioè circa il 6%. Le baseline descrivono ogni voto solo con tendenze "medie". Il modello migliore, `user_item_bias`, dice che il voto di un utente a un film è circa la media globale, più quanto l'utente è generoso (b_u), più quanto il film è apprezzato in generale (b_i).

Manca però il gusto personale. **Due utenti ugualmente generosi possono amare generi diversi**, e le baseline non hanno modo di saperlo. Il punto 3 serve a catturare proprio questa interazione specifica tra utente e film.

2 Il modello

2.1 · I fattori latenti

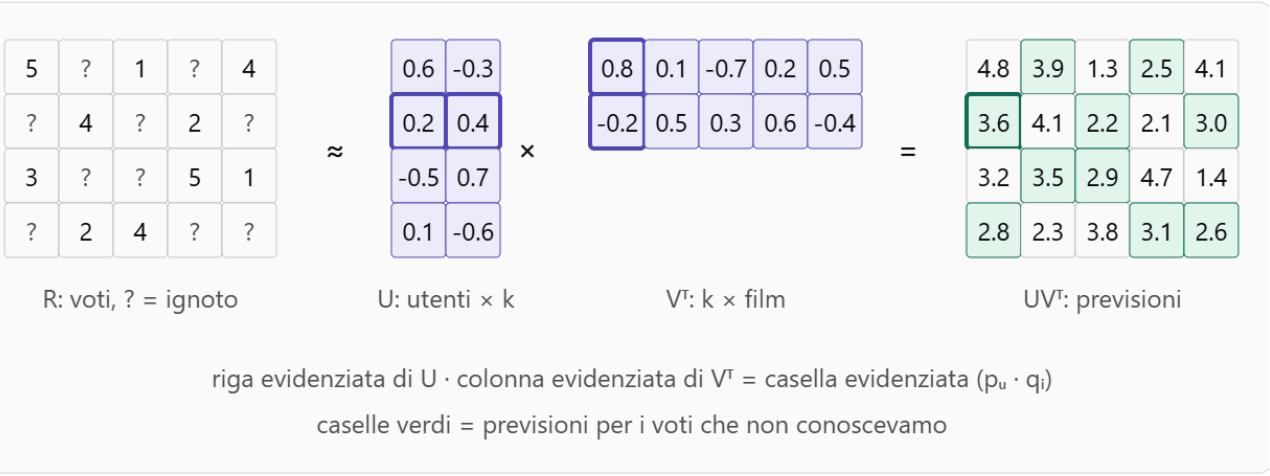
L'idea della fattorizzazione è **descrivere ogni utente e ogni film con un piccolo vettore di k numeri**, detti **fattori latenti**. Per capirlo, immaginiamo $k = 2$ con due dimensioni interpretabili, per esempio "quanto è un film d'azione" e "quanto è un film romantico". Un film d'azione puro avrà un vettore tipo $q_i = (0.9, -0.2)$. Un utente che ama l'azione e detesta i romantici avrà $p_u = (0.8, -0.5)$. Il prodotto scalare $p_u \cdot q_i = 0.9 \cdot 0.8 + (-0.2)(-0.5) = 0.82$ è alto: utente e film "si somigliano", quindi ci aspettiamo un voto sopra la media.



Nella realtà il modello *non sa cosa siano "azione" o "romantico"*. **Trova da solo k direzioni che spiegano bene i voti**, ed è per questo che si chiamano latenti: sono nascoste, apprese dai dati e non necessariamente interpretabili.

2.2 · La fattorizzazione a rango basso

Se mettiamo tutti i vettori degli utenti come righe di una matrice U (943 × k) e quelli dei film in una matrice V (1682 × k), **il prodotto UV^T dà una matrice 943 × 1682 con una previsione per ogni casella, anche per quelle vuote**. Con k = 10 i parametri sono (943 + 1682) × 10 ≈ 26.000 invece di 1,6 milioni. Per questo si parla di **fattorizzazione a rango basso**: il rango di UV^T è al massimo k.



Esempio: la matrice dei voti R (con caselle ignote) è approssimata dal prodotto U × V^T, che fornisce una previsione per ogni casella.

2.3 · Il modello completo: bias e fattori latenti

Il codice unisce le due idee, bias del punto precedente e fattori latenti:

$$\hat{r}_{ui} = \mu + b_u + b_i + p_u \cdot q_i$$

- μ è la media globale dei voti del training.
- b_u e b_i sono i bias di utente e film, come nel punto precedente.
- $p_u \cdot q_i$ è il **prodotto scalare tra i vettori latenti**.

PERCHÉ TENERE I BIAS SEPARATI

Tenere i bias separati dai fattori latenti permette di **dividere i compiti**: i bias spiegano la parte "facile" del voto (l'utente generoso, il film famoso), mentre i fattori latenti si concentrano solo sull'affinità di gusti. Senza bias, i fattori dovrebbero sprecare parte delle loro k dimensioni per imitare la generosità degli utenti. La traccia indica l'aggiunta dei bias come possibile estensione del modello base, e **il nostro modello la include già**. È il modello proposto da Koren, Bell e Volinsky [1].

3 L'addestramento con SGD

3.1 · Funzione obiettivo: cosa vogliamo minimizzare

Usiamo l'errore quadratico, sommato **solo sui voti osservati del training** (insieme K):

ERRORE SUI VOTI OSSERVATI + REGOLARIZZAZIONE

$$\min \Sigma_{(u,i) \in K} (r_{ui} - \hat{r}_{ui})^2 + \lambda \left(b_u^2 + b_i^2 + \|p_u\|^2 + \|q_i\|^2 \right)$$

Il primo termine misura quanto le previsioni si discostano dai voti veri. Il fatto che la somma corra solo su K è **il punto centrale di tutto il progetto**: le caselle vuote non compaiono mai nella funzione obiettivo, quindi il modello non viene mai spinto a prevedere zero dove non sa nulla.

Il secondo termine è la **regolarizzazione** (L2, detta anche ridge). Penalizza parametri troppo grandi. Senza questo termine, un utente con soli 20 voti potrebbe avere un vettore p_u enorme, costruito su misura per riprodurre esattamente quei 20 voti: errore zero sul training, previsioni pessime sul resto. È l'**overfitting**.

λ grande	λ giusto	λ piccolo
parametri vicini a zero → simile alla baseline	compromesso, scelto nel punto 4	modello libero → rischio di imparare il rumore

λ regola il compromesso: con λ grande i parametri restano vicini a zero e il modello torna a somigliare alla baseline; con λ piccolo il modello è libero ma rischia di imparare il rumore. Trovare il λ giusto è il compito del punto successivo.

3.2 · SGD (Stochastic Gradient Descent)

La funzione obiettivo dipende da decine di migliaia di parametri e **non ha una formula chiusa per il minimo**, perché p_u e q_i si moltiplicano tra loro. Il gradiente di una funzione indica la direzione in cui la funzione cresce più rapidamente. **Muovendo i parametri nella direzione opposta del gradiente, l'errore diminuisce.** Ripetendo il passo molte volte si scende verso un minimo. Questo metodo si chiama **discesa del gradiente**. Noi ne usiamo la versione stocastica, detta SGD (Stochastic Gradient Descent).

PERCHÉ "STOCASTICA"

La discesa del gradiente classica calcola il gradiente sull'intera somma degli 80.000 voti prima di fare un solo passo. **Lo SGD fa un passo dopo ogni singolo voto**, guardando solo il termine della somma che riguarda quel voto. Ogni passo è un po' impreciso, ma i passi sono velocissimi e nel complesso la discesa funziona molto bene.

3.3 · Derivazione delle regole di aggiornamento

Prendiamo un singolo voto r_{ui} e chiamiamo $e_{ui} = r_{ui} - \hat{r}_{ui}$ l'**errore di previsione**. La parte di funzione obiettivo che lo riguarda è:

$$L = e_{ui}^2 + \lambda \left(b_u^2 + b_i^2 + \|p_u\|^2 + \|q_i\|^2 \right)$$

Deriviamo rispetto a ciascun parametro, ricordando che $\hat{r}$ contiene b_u con coefficiente 1 e p_u moltiplicato per q_i, quindi $\partial e_{ui}/\partial b_u = -1$ e $\partial e_{ui}/\partial p_u = -q_i$:

GRADIENTI

$$\partial L/\partial b_u = -2e_{ui} + 2\lambda b_u$$
$$\partial L/\partial b_i = -2e_{ui} + 2\lambda b_i$$
$$\partial L/\partial p_u = -2e_{ui}\, q_i + 2\lambda p_u$$
$$\partial L/\partial q_i = -2e_{ui}\, p_u + 2\lambda q_i$$

Il passo è "parametro ← parametro − η · gradiente", dove **η è il learning rate**, cioè la lunghezza del passo. Il fattore 2 si assorbe dentro η, e si ottengono **le quattro regole che troviamo nel codice**:

REGOLE DI AGGIORNAMENTO SGD

$$b_u \leftarrow b_u + \eta \left(e_{ui} - \lambda\, b_u \right)$$
$$b_i \leftarrow b_i + \eta \left(e_{ui} - \lambda\, b_i \right)$$
$$p_u \leftarrow p_u + \eta \left(e_{ui}\, q_i - \lambda\, p_u \right)$$
$$q_i \leftarrow q_i + \eta \left(e_{ui}\, p_u - \lambda\, q_i \right)$$

COME SI LEGGONO

Se il modello ha previsto troppo poco ($e_{ui} > 0$), i bias aumentano e p_u si sposta un po' verso q_i (e viceversa), così il loro prodotto scalare cresce e **la prossima previsione sarà più alta**. Il termine $-\lambda$ · parametro tira invece sempre tutto un po' verso zero: è **la regolarizzazione che agisce a ogni passo**.

3.4 · Il sottoproblema con un fattore fissato

Il problema completo è difficile perché la previsione contiene il prodotto $p_u \cdot q_i$, cioè **due incognite che si moltiplicano**. Il "paesaggio" dell'errore ha quindi più valli (in termini tecnici, il problema **non è convesso**), e quella in cui si finisce dipende dal punto di partenza: è per questo che i valori iniziali casuali, e quindi il seed, influenzano il risultato. **Il problema diventa molto più semplice se si blocca uno dei due fattori.**

Supponiamo di conoscere già i vettori di tutti i film q_i e di dover trovare solo il vettore p_u di un utente. Il problema diventa:

$$\min_{p_u} \Sigma_{i \in I_u} (r_{ui} - p_u \cdot q_i)^2 + \lambda \|p_u\|^2$$

dove la somma riguarda solo i film votati dall'utente u. I bias si possono ignorare (basta sottrarli prima dal voto). Questo problema è una normale **regressione lineare con regolarizzazione (regressione ridge)**: cerchiamo i coefficienti p_u che, moltiplicati per i q_i noti, riproducono meglio i voti dell'utente. **A differenza del problema completo, qui c'è un unico minimo.** Lo stesso vale al contrario: se si bloccano gli utenti, trovare il vettore di un film è di nuovo una regressione ridge.

3.5 · Due modi di risolverlo: SGD e ALS

SGD. Il codice usa proprio questa idea, ma un voto alla volta. Quando aggiorna p_u tiene fermo q_i (è il motivo della `.copy()`), e la regola $p_u \leftarrow p_u + \eta(e_{ui}\, q_i - \lambda p_u)$ è un piccolo passo verso il minimo di questo sottoproblema. Poi fa lo stesso per q_i tenendo fermo p_u. Ripetendo questi piccoli passi su tutti i voti e per più epoche, utenti e film migliorano insieme.

ALS (Alternating Least Squares). Invece di fare piccoli passi, si può risolvere il sottoproblema in un colpo solo, con la formula esatta della regressione ridge:

$$p_u = (Q_u{}^T Q_u + \lambda I)^{-1}\, Q_u{}^T\, r_u$$

Q_u contiene i vettori dei film votati dall'utente u e r_u i suoi voti. L'ALS calcola questa formula per tutti gli utenti tenendo fermi i film, poi fa il contrario, e alterna fino a convergenza. Il termine λI garantisce che la matrice da invertire sia sempre invertibile, anche per utenti con pochi voti. L'ALS ha alcuni vantaggi: non serve scegliere un learning rate, converge in poche iterazioni e in modo regolare, e il calcolo di ogni utente è indipendente dagli altri, quindi si può parallelizzare. Lo svantaggio è che ogni iterazione richiede di risolvere un sistema lineare (invertire una matrice k × k) per ogni utente e per ogni film, e che i bias vanno gestiti con passaggi aggiuntivi.

	SGD	ALS
Come risolve il sottoproblema	tanti passi piccoli ed economici, un voto alla volta	esattamente, un pezzo del problema alla volta
Learning rate	da scegliere	non serve
Convergenza	da controllare (RMSE di validation)	poche iterazioni, regolare
Costo	basso per ogni passo	un sistema lineare k × k per ogni utente e film
Bias b_u e b_i	gestiti facilmente	passaggi aggiuntivi

SCELTA CONCLUSIVA DELL'ALGORITMO

L'ALS risolve esattamente un pezzo del problema alla volta, mentre lo SGD fa tanti passi piccoli ed economici, un voto alla volta. **Abbiamo scelto lo SGD perché è più semplice da implementare, gestisce facilmente i bias b_u e b_i, ed è il metodo proposto da Koren et al. [1] per i voti espliciti come quelli di MovieLens.** Il prezzo è dover scegliere anche il learning rate e controllare la convergenza, cosa che abbiamo fatto **monitorando l'RMSE di validation a ogni epoca**.

4 Analisi e interpretazione dei risultati

4.1 · Grid search

Rank (k)	Regularization (λ)	Best_Epoch	Validation_RMSE
10	**0.05**	**20**	**0.9245**
20	0.05	20	0.9248
5	0.05	20	0.9300
20	0.10	20	0.9307
10	0.10	20	0.9326
10	0.01	13	0.9348
5	0.01	13	0.9369
20	0.01	11	0.9385
5	0.10	20	0.9397

LA STESSA TABELLA COME MAPPA: PIÙ SCURO = RMSE PIÙ BASSO

k \ λ	0.01	0.05	0.1
k = 5	0.9369	0.9300	0.9397
k = 10	0.9348	0.9245	0.9326
k = 20	0.9385	0.9248	0.9307

bordo rosso = combinazione scelta

La tabella va letta **guardando un parametro alla volta**.

Effetto della regolarizzazione λ. Per ogni valore di k, **il λ migliore è sempre 0.05**.

- **λ = 0.01 (troppo debole).** L'epoca migliore arriva presto (11–13) e l'RMSE finale è più alto. Il modello impara il training molto in fretta, ma dopo qualche epoca comincia a imparare anche il rumore e la validation peggiora: è **overfitting**. Infatti è l'unico caso in cui l'early stopping interviene davvero prima della fine.
- **λ = 0.1 (troppo forte).** I parametri vengono tenuti troppo vicini a zero e il modello non riesce a sfruttare bene i fattori latenti: è **underfitting**. Qui l'epoca migliore è 20 perché il modello sta ancora migliorando lentamente.
- **λ = 0.05 è il compromesso tra i due estremi.**

Effetto del rango k. Passare da k = 5 a k = 10 aiuta (con λ = 0.05: da 0.9300 a 0.9245). Passare da 10 a 20 invece non porta nulla (0.9248). Su questo dataset **bastano circa 10 "dimensioni di gusto"** per

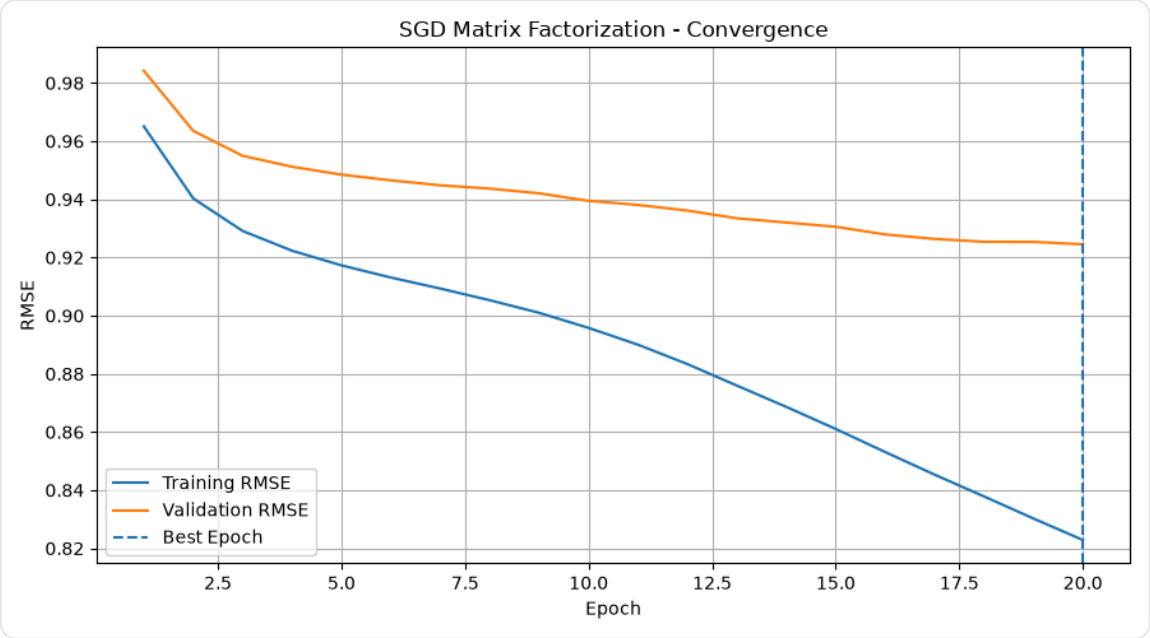
descrivere utenti e film: aggiungerne altre aumenta i parametri senza aggiungere informazione utile. A parità di risultato conviene il modello più semplice, quindi **k = 10 è una scelta ragionevole** anche al di là del piccolo vantaggio numerico.

4.2 · Il modello migliore

RANK	REGULARIZATION	BEST EPOCH	VALIDATION RMSE
10	0.05	20	0.9245

Tra la combinazione migliore (0.9245) e la peggiore (0.9397) ci sono circa 0.015 punti di RMSE. Conta, ma è molto meno del salto tra le baseline semplici e quelle con i bias. Questo significa che **la parte più grossa del lavoro la fanno già i bias**, e i fattori latenti aggiungono un miglioramento più fine.

4.3 · Curva di convergenza

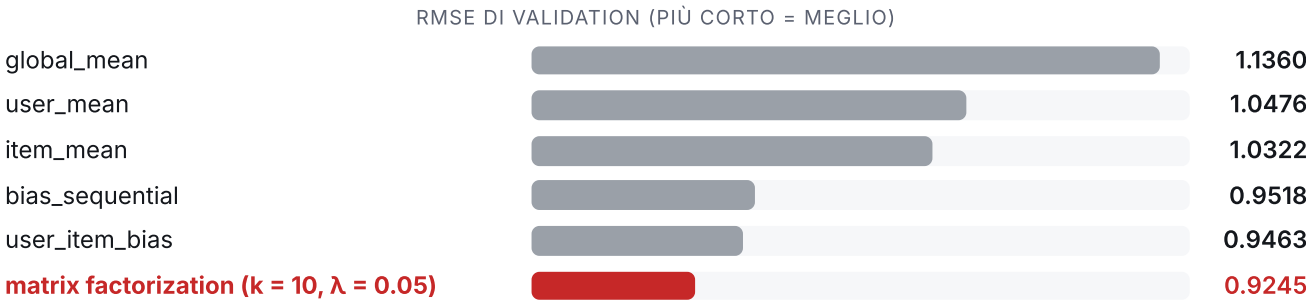


RMSE di training e di validation epoca per epoca per il modello migliore (k = 10, λ = 0.05).

- **La curva di training** scende per tutte le 20 epoche, da 0.965 a 0.823, e non accenna a fermarsi. È normale: è proprio la quantità che lo SGD sta minimizzando.
- **La curva di validation** scende rapidamente nelle prime 3–4 epoche, poi sempre più lentamente, e nelle ultime epoche è quasi piatta (0.9254, 0.9254, 0.9245).
- **La distanza tra le due curve cresce.** Al primo epoch è circa 0.02, all'ultimo circa 0.10. Il modello sta diventando sempre più bravo sui voti che ha visto, ma molto meno su quelli nuovi. È il segnale che **si sta avvicinando all'overfitting**.
- **La linea tratteggiata all'epoca 20** indica che l'epoca migliore è l'ultima disponibile: il limite di 20 epoche "taglia" la curva proprio mentre si appiattisce. Facendo girare lo stesso modello per 50 epoche: il minimo arriva all'epoca 23 (0.9235), poi la validation risale fino a 0.943 mentre il

training continua a scendere. Quindi **20 epoche sono quasi il punto giusto** e il guadagno perso è minimo, ma il grafico, guardato da solo, non lo dimostra.

4.4 · Confronto con la baseline del punto precedente



La fattorizzazione migliora la migliore baseline del 2,3%.

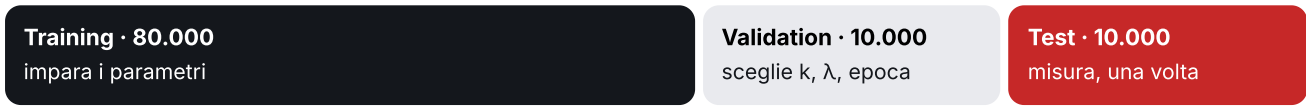
Risultati del modello migliore sulla validation e alcuni esempi di previsione:

Metrica	Valore
RMSE	0.9245
MAE	0.7281
R2	0.3375
Accuracy	42.1100%
Within_1	89.8700%

user_id	movie_id	rating	prediction
868	470	1	3.01
472	385	5	4.64
928	487	5	4.79
605	288	5	3.93
751	734	1	2.65

5 Valutazione finale sul test set

5.1 · Perché serve il test set



Il training è servito a imparare i parametri (b_u, b_i, p_u, q_i). **La validation** è servita a scegliere k, λ e l'epoca migliore: tra 9 modelli, ciascuno controllato per 20 epoche, abbiamo tenuto quello con l'RMSE di validation più basso. Se proviamo molte combinazioni e teniamo la migliore sulla validation, **quel numero (0.9245) è un po' ottimistico**.

Il test set risolve questo problema. Sono 10.000 voti che il modello non ha mai usato: né per imparare i parametri, né per scegliere gli iperparametri. Il risultato sul test è quindi una stima onesta di come il modello si comporterebbe su voti nuovi. **È l'esame vero, dopo le simulazioni.**

5.2 · Evaluation leakage

Evaluation leakage: la "fuga" di informazione dal test verso le scelte del modello. Succede quando il test viene usato, anche indirettamente, per prendere decisioni. Per esempio:

- scegliere k guardando l'RMSE sul test;
- provare varie versioni del codice finché il test migliora;
- calcolare le medie delle baseline usando anche i voti del test.

Il nostro codice evita il leakage perché:

✓ tutte le scelte avvengono in `best_hyperparameters(train, val)`, che non riceve il test;

✓ anche le baseline sono addestrate solo su train;

✓ il test entra in gioco solo alla fine, una volta, per misurare.

5.3 · Risultati sul test

Sul test set, mai usato prima, si ottiene questo:

Modello	RMSE	MAE	R^2	Within_1
Matrix factorization	**0.9213**	**0.7274**	**0.345**	**90,0%**
user_item_bias	0.9458	0.7511	0.310	89,1%
item_mean	1.0321	0.8273	0.178	86,1%
global_mean	1.1384	0.9582	0.000	82,0%

L'RMSE sul test (0.9213) è **molto vicino a quello di validation (0.9245)**, anzi leggermente migliore. **È la conferma più importante del punto 5.** Vuol dire che $k = 10$ e $\lambda = 0.05$ non erano una scelta "fortunata" su quei particolari 10.000 voti di validation: **il modello generalizza davvero a voti nuovi**. La piccola differenza a favore del test è normale variabilità tra due campioni, non un miglioramento reale.

Anche le baseline hanno risultati quasi identici a quelli del punto 2 (`user_item_bias`: 0.9463 in validation, 0.9458 sul test) e mantengono lo stesso ordine. **Tutto il quadro è stabile.**

5.4 · Conclusioni

1 **La maggior parte del lavoro la fanno già i bias.** Sapere che un utente è generoso e che un film è molto apprezzato spiega gran parte dei voti. I fattori latenti aggiungono un miglioramento più piccolo ma reale: catturano l'interazione tra utente e film, cioè il fatto che a quell'utente piaccia quel tipo di film. È un risultato tipico in letteratura (anche Koren et al. osservano che i bias spiegano molto del segnale), quindi va presentato così, non come un fallimento.

2 **Il miglioramento è inoltre diffuso, non concentrato su pochi voti:** la fattorizzazione si avvicina al voto vero più della baseline nel **54% dei casi**, e gli errori gravi (2 o più stelle) scendono dal 3,6% al 3,3%.

3 **Un errore più basso sul training non basta.** Durante l'addestramento l'RMSE di training scende fino a 0.82, mentre quello di validation si ferma a 0.92. **Ridurre l'errore sui voti osservati oltre un certo punto non migliora le previsioni su voti nuovi.**

RIFERIMENTI

[1] Y. Koren, R. Bell, C. Volinsky, "Matrix Factorization Techniques for Recommender Systems," *Computer*, 42(8), 2009. doi:10.1109/MC.2009.263.